

Chapter 8: Applets and Application

8.1. Applet Fundamentals

Applets are small applications that are accessed on an Internet server, transported over the Internet, automatically installed, and run as part of a web document. After an applet arrives on the client, it has limited access to resources so that it can produce a graphical user interface and run complex computations without introducing the risk of viruses or breaching data integrity. Applets differ from console-based applications in several key areas.

```
import java.awt.*;
import java.applet.*;
public class SimpleApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("A Simple Applet", 20, 20);
    }
}
```

1. This applet begins with two **import** statements. The first imports the *Abstract Window Toolkit* (AWT) classes. Applets interact with the user (either directly or indirectly) through the AWT, not through the console-based I/O classes. The AWT contains support for a window-based, graphical user interface.
2. The second **import** statement imports the **applet** package, which contains the class **Applet**. Every applet that we create must be a subclass of **Applet**.
3. The next line in the program declares the class **SimpleApplet**. This class must be declared as **public**, because it will be accessed by code that is outside the program.
4. Inside **SimpleApplet**, **paint()** is declared. This method is defined by the AWT and must be overridden by the **applet**. **paint()** is called each time that the applet must redisplay its output. This situation can occur for several reasons. For example, *the window in which the applet is running can be overwritten by another window and then uncovered. Or, the applet window can be minimized and then restored.* **paint()** is also called when the applet begins execution. Whatever the cause, whenever the applet must redraw its output, **paint()** is called. The **paint()** method has one parameter of type **Graphics**. This parameter contains the graphics context, which describes the graphics environment in which the applet is running. This context is used whenever output to the applet is required.
5. Inside **paint()** is a call to **drawString()**, which is a member of the **Graphics** class. This method outputs a string beginning at the specified X,Y location. It has the following general form:
void drawString(String message, int x, int y)
6. Here, **message** is the string to be output beginning at x,y. In a Java window, the upper-left corner is location 0,0. The call to **drawString()** in the applet causes the message "A Simple Applet" to be displayed beginning at location 20,20.
7. Notice that the applet **does not have a main()** method. Unlike Java programs, applets do not begin execution at **main()**. In fact, most applets don't even have a **main()** method. Instead, an applet begins execution **when the name of its class is passed to an applet viewer** or to a network browser.
8. After we enter the source code for **SimpleApplet**, compile in the same way that we have been compiling programs. However, running **SimpleApplet** involves a different process. In fact, there are two ways in which you can run an applet:
 - Executing the applet within a Java-compatible web browser.
 - Using an applet viewer, such as the standard tool, **appletviewer**. An applet viewer executes our applet in a window. This is generally the fastest and easiest way to test our applet.

To execute an applet in a web browser, we need to write a short HTML text file that contains a tag that loads the applet. Currently, Sun recommends using the **APPLET** tag for this purpose. Here is the HTML

file that executes *SimpleApplet*:

```
<applet code="SimpleApplet" width=200 height=60>
</applet>
```

The width and height statements specify the dimensions of the display area used by the applet. After the creation of this file, we can execute our browser and then load this file, which causes *SimpleApplet* to be executed.

To execute *SimpleApplet* with an applet viewer, we may also execute the HTML file shown earlier. For example, if the preceding HTML file is called *RunApp.html*, then the following command line will run *SimpleApplet*:

```
C:\>appletviewer RunApp.html
```

However, a more convenient method exists that we can use to speed up testing. Simply include a comment at the head of Java source code file that contains the APPLET tag. By doing so, our code is documented with a prototype of the necessary HTML statements, and we can test our compiled applet merely by starting the applet viewer with our Java source code file. The *SimpleApplet* source file looks like this:

```
import java.awt.*;
import java.applet.*;
/*
<applet code="SimpleApplet" width=200 height=60>
</applet>
*/
public class SimpleApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("A Simple Applet", 20, 20);
    }
}
```

Key Points

1. Applets do not need a `main()` method.
2. Applets must be run under an applet viewer or a Java-compatible browser.
3. User I/O is not accomplished with Java's stream I/O classes. Instead, applets use the interface provided by the AWT or Swing.
4. All applets are subclasses (either directly or indirectly) of `Applet`.
5. Applets are not stand-alone programs. Instead, they run within either a web browser or an applet viewer.
6. Output is handled with various AWT methods, such as *`drawString()`*, which outputs a string to a specified X,Y location, not by *`system.out.println()`*.
7. A Java-enabled web browser will execute the applet when it encounters the APPLET tag within the HTML file.

8.2. Two Types of Applet

There are two varieties of applets.

1. The first type of applets uses the ***Abstract Window Toolkit (AWT)*** to provide the graphic user interface (or use no GUI at all). This style of applet has been available since Java was first created.
2. The second type of applets is those based on the Swing class ***JApplet***. Swing applets use the Swing classes to provide the GUI. *Swing offers a richer and often easier-to-use user interface than does the AWT*. Thus, Swing-based applets are now the most popular. However, traditional AWT-based applets are still used, especially when only a very simple user interface is required. Thus, both AWT- and Swing-based applets are valid. Because ***JApplet*** inherits ***Applet***, all the features of ***Applet*** are also available in ***JApplet***.

8.3. The Applet Class

The **Applet** class defines the methods shown in following table. **Applet** provides all necessary support for applet execution, such as *starting and stopping*. It also provides methods that *load and display images*, and *methods that load and play audio clips*. **Applet** extends the **AWT class Panel**. In turn, **Panel** extends **Container**, which extends **Component**. These classes provide support for Java's window-based, graphical interface. Thus, **Applet** provides all of the necessary support for window-based activities.

Methods	Description
<code>void destroy()</code>	Called by the browser just before an applet is terminated.
<code>AccessibleContext getAccessibleContext()</code>	Returns the accessibility context for the invoking object.
<code>AppletContext getAppletContext()</code>	Returns the context associated with the applet.
<code>String getAppletInfo()</code>	Returns a string that describes the applet.
<code>AudioClip getAudioClip(URL url)</code>	Returns an AudioClip object that encapsulates the audio clip found at the location specified by url.
<code>URL getCodeBase()</code>	Returns the URL associated with the invoking applet.
<code>Image getImage(URL url)</code>	Returns an Image object that encapsulates the image found at the location specified by url.
<code>Locale getLocale()</code>	Returns a Locale object that is used by various locale sensitive classes and methods.
<code>String getParameter(String paramName)</code>	Returns the parameter associated with paramName . null is returned if the specified parameter is not found.
<code>void init()</code>	Called when an applet begins execution. It is the first method called for any applet.
<code>boolean isActive()</code>	Returns true if the applet has been started. It returns false if the applet has been stopped.
<code>void play(URL url)</code>	If an audio clip is found at the location specified by url, the clip is played.
<code>void play(URL url, String clipName)</code>	If an audio clip is found at the location specified by url with the name specified by clipName, the clip is played.
<code>void start()</code>	Called by the browser when an applet should start (or resume) execution. It is automatically called after <code>init()</code> when an applet first begins.
<code>void stop()</code>	Called by the browser to suspend execution of the applet. Once stopped, an applet is restarted when the browser calls <code>start()</code> .
<code>void showStatus(String str)</code>	Displays str in the status window of the browser or applet viewer. If the browser does not support a status window, then no action takes place.

And many more.. (refer to text book)

Table: The Methods Defined by Applet

8.4. Applet Architecture

An applet is a window-based program. As such, its architecture is different from the console-based programs. First, applets are event driven. It is important to understand in a general way that the event-driven architecture impacts the design of an applet.

An applet resembles a set of *interrupt service routines*. Here is how the process works. An applet waits until an **event** occurs. The **run-time system** notifies the applet about an event by calling an event handler that has been provided by the applet. Once this happens, the applet must take appropriate action and then quickly return. This is a crucial point. For the most part, applet should not enter a "mode" of operation in which it maintains control for an extended period. Instead, *it must perform specific actions in response to events and then return control to the run-time system*. In those situations in which applet needs to perform a repetitive task on its own (for example, displaying a scrolling message across its window), we must start an additional thread of execution.

Second, the user initiates interaction with an applet. These interactions are sent to the applet *as events to which the applet must respond*. For example, when the user clicks the mouse inside the applet's window, a mouse-clicked event is generated. If the user presses a key while the applet's window has input focus, a *keypress* event is generated. **Applets** can contain various controls, such as push buttons and check boxes. When the user interacts with one of these controls, an event is generated.

8.5. Applet Life Cycle Methods

There are five applet methods that are called by the applet container from the time the applet is loaded into the browser to the time it's terminated by the browser. These methods correspond to various aspects of an applet's life cycle.

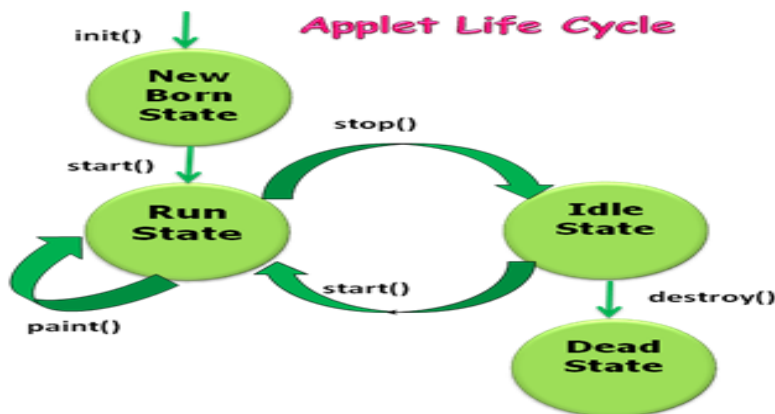


Fig: Applet Life Cycle

Methods	Description
<code>public void init()</code>	Called once by the applet container when an applet is loaded for execution. This method initializes an applet. Typical actions performed here are initializing fields, creating GUI components, loading sounds to play, loading images to display and creating threads.
<code>public void start()</code>	Called by the applet container after method init completes execution. In addition, if the user browses to another website and later returns to the applet's HTML page, method start is called again. The method performs any tasks that must be completed when the applet is loaded for the first time and that must be performed every time the applet's HTML page is revisited. Actions performed here might include starting an animation or starting other threads of execution.
<code>public void paint(Graphics g)</code>	Called by the applet container after methods init and start . Method paint is also called when the applet needs to be repainted. For example, if the user covers the applet with another open window on the screen and later uncovers it, the paint method is called. Typical actions performed here involve drawing with the Graphics object g that's passed to the paint method by the applet container.
<code>public void stop()</code>	The applet container calls this method when the user leaves the applet's web page by browsing to another web page. Since it's possible that the user might return to the web page containing the applet , method stop performs tasks that might be required to suspend the applet's execution, so that the applet does not use computer processing time when it's not displayed on the screen. Typical actions performed here would stop the execution of animations and threads.
<code>public void destroy()</code>	The applet container calls this method when the applet is being removed from memory. This occurs when the user exits the browsing session by closing all the browser windows and may also occur at the browser's discretion when the user has browsed to other web pages. The method performs any tasks that are required to clean up resources allocated to the applet.

8.6. An Applet Skeleton

Four of these methods, **init()**, **start()**, **stop()**, and **destroy()**, apply to all applets and are defined by Applet. Default implementations for all of these methods are provided.

```
// An Applet skeleton.
import java.awt.*;
import java.applet.*;
/*
<applet code="AppletSkel" width=300 height=100>
</applet>
*/
public class AppletSkel extends Applet {
    // Called first.
    public void init() {
        // initialization
    }
    /* Called second, after init(). Also called whenever the applet is restarted. */
    public void start() {
        // start or resume execution
    }
    // Called when the applet is stopped.
    public void stop() {
        // suspends execution
    }
    /* Called when applet is terminated. This is the last method executed. */
    public void destroy() {
        // perform shutdown activities
    }
    // Called when an applet's window must be restored.
    public void paint(Graphics g) {
        // redisplay contents of window
    }
}
```

8.7. The HTML APPLET Tag

The APPLET tag be used to start an applet from both an HTML document and from an applet viewer. An applet viewer will execute each APPLET tag that it finds in a separate window, while web browsers will allow many applets on a single page. The syntax for a fuller form of the APPLET tag is shown here. Bracketed items are optional while **code**, **width** and **height** attributes are required.

```
<APPLET
[CODEBASE = codebaseURL]
CODE = appletFile
[ALT = alternateText]
[NAME = appletInstanceName]
WIDTH = pixels
HEIGHT = pixels
[ALIGN = alignment]
[VSPACE = pixels] [HSPACE = pixels]
>
[< PARAM NAME = AttributeName VALUE = AttributeValue>]
[< PARAM NAME = AttributeName2 VALUE = AttributeValue>]
...
[HTML Displayed in the absence of Java]
</APPLET>
```

- **CODEBASE** is an optional attribute that specifies the base URL of the applet code, which is the directory that will be searched for the applet's executable class file. The HTML document's URL directory is used as the CODEBASE if this attribute is not specified. The CODEBASE does not have to be on the host from which the HTML document was read.
- **CODE** is a required attribute that gives the name of the file containing applet's compiled **.class** file. This file is relative to the code base URL of the applet, which is the directory that the HTML file was in or the directory indicated by CODEBASE if set.
- **ALT** The ALT tag is an optional attribute used to specify a short text message that should be displayed if the browser recognizes the APPLET tag but can't currently run Java applets.
- **NAME** is an optional attribute used to specify a name for the applet instance. Applets must be named in order for other applets on the same page to find them by name and communicate with them. To obtain an applet by name, use *getApplet()*, which is defined by the *AppletContext* interface.
- **WIDTH** and **HEIGHT** are required attributes that give the size (in pixels) of the applet display area.
- **ALIGN** is an optional attribute that specifies the alignment of the applet. This attribute is treated the same as the HTML IMG tag with these possible values: LEFT, RIGHT, TOP, BOTTOM, MIDDLE, BASELINE, TEXTTOP, ABSMIDDLE, and ABSBOTTOM.
- **VSPACE** and **HSPACE** are optional attributes. VSPACE specifies the space, in pixels, above and below the applet. HSPACE specifies the space, in pixels, on each side of the applet. They're treated the same as the IMG tag's VSPACE and HSPACE attributes.
- **PARAM NAME** and **VALUE** The PARAM tag allows us to specify applet-specific arguments in an HTML page. Applets access their attributes with the *getParameter()* method.

8.8. Passing Parameters to Applets

The **APPLET** tag in HTML allows us to pass parameters to our applet. To retrieve a parameter, we need to use the *getParameter()* method. It returns the value of the specified parameter in the form of a **String** object. Thus, for **numeric** and **boolean** values, we will need to convert their string representations into their internal formats.

```
// Use Parameters
/*
<applet code="ParamDemo" width="400" height="100">
<param name="fontName" value="Times new Roman">
<param name="fontSize" value="15">
<param name="leading" value="3">
<param name="accountEnabled" value="true">
</applet>
*/
```

```
import java.awt.*;
import java.applet.*;

public class ParamDemo extends Applet{
String fontName;
int fontSize;
float leading;
boolean active;
// Initialize the string to be displayed.
public void start() {
    String param;
    fontName = getParameter("fontName");
    if(fontName == null)
```

```
fontName = "Not Found";
param = getParameter("fontSize");
try {
    if(param != null) // if not found
        fontSize = Integer.parseInt(param);
    else
        fontSize = 0;
} catch(NumberFormatException e) {
    fontSize = -1;
}
param = getParameter("leading");
try {
    if(param != null) // if not found
        leading = Float.valueOf(param).floatValue();
    else
        leading = 0;
} catch(NumberFormatException e) {
    leading = -1;
}
param = getParameter("accountEnabled");
if(param != null)
    active = Boolean.valueOf(param).booleanValue();
}
// Display parameters.
public void paint(Graphics g) {
    g.drawString("Font name: " + fontName, 0, 10);
    g.drawString("Font size: " + fontSize, 0, 26);
    g.drawString("Leading: " + leading, 0, 42);
    g.drawString("Account Active: " + active, 0, 58);
}
}
```

8.9. Applet Security Basics

Because applets are designed to be loaded from a remote site and then executed locally, security becomes vital. If a user enables Java in the browser, the browser will download all the applet code on the web page and execute it immediately. The user never gets a chance to confirm or to stop individual applets from running. For this reason, applets (unlike applications) are restricted in what they can do. The applet security manager throws a ***SecurityException*** whenever an applet attempts to violate one of the access rules. What can applets do on all platforms? They can show images and *play sounds, get keystrokes and mouse clicks from the user, and send user input back to the host* from which they were loaded. That is enough functionality to show facts and figures or to get user input for placing an order. The restricted execution environment for applets is often called the “sandbox.” Applets playing in the “sandbox” cannot alter the user's system or spy on it.

In particular, when running in the sandbox:

- Applets can never run any local executable program.
- Applets cannot communicate with any host other than the server from which they were downloaded; that server is called the ***originating host***. This rule is often called “applets can only phone home.” This protects applet users from applets that might try to spy on intranet resources.
- Applets cannot read from or write to the local computer's file system.
- Applets cannot find out any information about the local computer, except for the Java version used, the name and version of the operating system, and the characters used to separate files (for instance, / or \), paths (such as : or ;), and lines (such as \n or \r\n). In particular, applets cannot find out the user's name, e-mail address, and so on.

- All windows that an applet pops up carry a warning message.

All this is possible only because applets are interpreted by the Java Virtual Machine and not directly executed by the CPU on the user's computer. Because the interpreter checks all critical instructions and program areas, a hostile (or poorly written) applet will almost certainly not be able to crash the computer, overwrite system memory, or change the privileges granted by the operating system.

These restrictions are too strong for some situations. For example, on a corporate intranet, we can certainly imagine an applet wanting to access local files. To allow for different levels of security under different situations, we can use **signed applets**. A signed applet carries with it a certificate that indicates the identity of the signer. Cryptographic techniques ensure that such a certificate cannot be forged. If we trust the signer, we can choose to give the applet additional rights.

The point is that if we trust the signer of the applet, we can tell the browser to give the applet more privileges. The **configurable Java security model** allows the continuum of privilege levels we need. We can give completely trusted applets the same privilege levels as local applications. Programs from vendors that are known to be somewhat flaky can be given access to some, but not all, privileges. Unknown applets can be restricted to the sandbox.

To sum up, Java has three separate mechanisms for enforcing security:

1. Program code is interpreted by the Java Virtual Machine, not executed directly.
2. A security manager checks all sensitive operations in the Java runtime library.
3. Applets can be signed to identify their origin.

8.10. Applet vs Application

Features	Application	Applet
main() method	Present	Not present
Execution	Requires JRE	Requires a browser or applet viewer
Nature	Called as stand-alone application as application can be executed from command prompt	Requires some third party tool help like a browser to execute
Restrictions	Can access any data or software available on the system	cannot access any thing on the system except browser's services
Security	Does not require any security	Requires highest security for the system as they are untrusted

8.11. Advantages of Applet

- Execution of applets is easy in a Web browser and does not require any installation or deployment procedure in real-time programming (where as servlets require).
- Writing and displaying (just opening in a browser) graphics and animations is easier than applications.
- In GUI development, constructor, size of frame, window closing code etc. are not required (but are required in applications).